UML Class Diagram

AI-Powered Student Planner and Task Management System | Django models and services

«package» Models (domain layer, Django models)

AIMessage
- message_id: int
- role: MessageRole
- content: string
- created_at: datetime

AIConversation
- conversation_id: int
- title: string
- started_at: datetime

+ addMessage(role, content): AIMessage
+ getHistory(): List<AIMessage>

User
- user_id: int
- username: string
- email: string
- password_hash: string
- first_name: string
- last_name: string
- ai_data_consent: bool
- created_at: datetime
- last_login: datetime

+ login(password): bool
+ logout(): void
+ updateEmail(newEmail): void
+ grantAIConsent(): void
+ revokeAIConsent(): void

StudyPlan
- plan_id: int
- title: string
- status: PlanStatus
- created_by_ai: bool
- created_at: datetime

+ activate(): void
+ discard(): void
+ getSessions(): List<StudySession>
+ hasConflicts(): bool

ClassSession
- class_id: int
- day_of_week: int
- start_time: time
- end_time: time
- location: string

+ overlaps(other): bool

Course
- course_id: int
- name: string
- code: string
- instructor: string
- semester: string
- color: string

+ rename(name): void
+ getTasks(): List<Task>

StudySession
- session_id: int
- start_at: datetime
- end_at: datetime
- status: SessionStatus
- actual_minutes: int
- source: CreationSource

+ complete(minutes): void
+ markMissed(): void
+ reschedule(start, end): void
+ cancel(): void

«abstract»
Task
- task_id: int
- title: string
- description: string
- due_at: datetime
- priority: Priority
- status: TaskStatus
- created_via: CreationSource
- created_at: datetime
- completed_at: datetime

+ updateStatus(status): void
+ setPriority(priority): void
+ markCompleted(): void
+ isOverdue(): bool
# validateDeadline(): bool

Assignment
- max_grade: decimal
- weight_percent: decimal

+ calculateWeightedScore(score): float

Exam
- exam_kind: ExamKind
- location: string
- duration_min: int

+ getDurationHours(): float

Reminder
- reminder_id: int
- remind_at: datetime
- status: ReminderStatus
- sent_at: datetime
- failure_reason: string

+ markSent(): void
+ markFailed(reason): void
+ cancel(): void

Legend

composition: parent owns its parts
aggregation: parts may exist alone
association (with multiplicity)
dependency (uses)
+ public - private # protected

«enumeration» types

Priority: LOW, MEDIUM, HIGH
TaskStatus: PENDING, IN_PROGRESS, COMPLETED, OVERDUE
CreationSource: MANUAL, AI
SessionStatus: PLANNED, COMPLETED, MISSED, CANCELLED
PlanStatus: DRAFT, ACTIVE, DISCARDED
ReminderStatus: SCHEDULED, SENT, FAILED, CANCELLED
MessageRole: USER, ASSISTANT
ExamKind: MIDTERM, FINAL, QUIZ

«uses»

«package» Services (application layer)

«service»
AuthService
manages: User
+ register(username, email, password): User
+ validateSession(token): int
+ logout(token): void

«service»
CourseService
manages: Course, ClassSession
+ addCourse(userId, data): Course
+ updateCourse(courseId, data): Course
+ deleteCourse(courseId): void

«service»
TaskService
manages: Task, Assignment, Exam
+ saveItem(userId, details): Task
+ createTask(userId, finalTask): Task
+ deleteItem(userId, itemId): void
+ resolveDates(taskDraft): TaskDraft
+ updateStatus(taskId, status): void

«service»
CalendarService
Jalali / Gregorian conversion
+ toJalali(date): string
+ fromJalali(text): date

«service»
ProgressService
reads: Task, StudySession
+ getStatistics(userId): Statistics
+ getStudyTime(userId, period): int

«service»
StudyPlanService
manages: StudyPlan, StudySession
+ createPlan(context, availableTime): StudyPlan
+ validate(plan): List<Conflict>
+ adjustPlan(planId): StudyPlan
+ saveStudySessions(userId, plan): void

«service»
AIService
manages: AIConversation, AIMessage
+ extractTaskDetails(text): TaskDraft
+ updateTaskDraft(draft, value): TaskDraft
+ generatePlan(context, prompt): StudyPlan
+ analyzeDeadlines(userId): string
+ getInsights(userId): string

«service»
ReminderService
manages: Reminder
+ processDueReminders(now): void
+ scheduleReminder(taskId, deadline): Reminder
+ refreshReminder(taskId, deadline): void
+ cancelReminder(taskId): void

«external service»
NotificationService
in-app notification channel
+ sendNotification(userId, message): bool